


**UNIVERSIDADE ESTADUAL DE SANTA CRUZ - UESC**

**CIÊNCIA DA COMPUTAÇÃO**

**LAISA PEREIRA FRANÇA**

**LUCAS GABRIEL FERREIRA**

# ETAPA 2 — BENCHMARK DE GERAÇÃO DO CONJUNTO DE MANDELBROT

**Escape-Time: Implementação paralela em memória distribuída com MPI.**

ILHÉUS – BA

2026

**LAISA PEREIRA FRANÇA**

**LUCAS GABRIEL FERREIRA**

# ETAPA 2 — BENCHMARK DE GERAÇÃO DO CONJUNTO DE MANDELBROT

## Escape-Time: Implementação paralela em memória distribuída com MPI

Estudo apresentado como requisito parcial à disciplina de Processamento Paralelo, DEC107, da Universidade Estadual de Santa Cruz, pelo curso de Ciência da Computação, turma de 2026.2.

Orientador (a): Prof. Esbel Tomas Valero Orellana.

ILHÉUS – BA

2026

**SUMÁRIO**

**1. Introdução..... 5**
**2. Metodologia..... 6**
**3. Resultados..... 9**
3.1 Teste de Corretude.....9
3.2 Caso Base - Tempo de execução serial.....9
3.3 Casos de Teste Estratégia/Chunk.....10
3.3.1 Representação Gráfica.....10
Speedup e Eficiência.....11
Carga mínima, média e máxima..... 12
Fator de Balanceamento de Carga..... 13
Custo de Comunicação..... 14
3.4 Casos de Estresse..... 15
3.4.1 Escalabilidade Forte (Casos de Teste)..... 15
3.4.1.1 Representação Gráfica.....16
Speedup e Eficiência.....16
Fator de Balanceamento de Carga..... 17
Custo de Comunicação..... 18
3.4.2 Desbalanceamento (Casos de Teste).....18
3.4.2.1 Representação Gráfica.....19
Speedup e Eficiência.....19
Carga mínima, média e máxima..... 21
Fator de Balanceamento de Carga..... 22
Custo de Comunicação..... 23
3.4.3 Escalabilidade Fraca (Casos de Teste)..... 23
3.4.3.1 Representação Gráfica.....24
Speedup e Eficiência.....25
Fator de Balanceamento de Carga..... 27
3.4.4 Comparação com a solução OpenMP.....27
**4. Discussão.....29**
4.1 Análise da Corretude da Implementação.....29
4.2 Comparação entre a Versão Sequencial e a Versão Paralela em MPI.....29
4.3 Comparação entre as Estratégias de Distribuição.....29
4.4 Análise do Impacto do Número de Processos no Desempenho.....31
4.5 Discussão de Gargalos Encontrados e Limitações da Implementação.....32
4.6 Comparação com a Solução OpenMP.....33
**5. Conclusão..... 35**
**6. Referências..... 36**

**DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL..... 37**
**DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL..... 38**

## 1. Introdução

O presente relatório tem como objetivo apresentar o desenvolvimento, o benchmark e a análise da paralelização da geração do Conjunto de Mandelbrot na Etapa 2 do projeto. Nesta etapa, o foco é a implementação de uma solução em C para arquitetura de memória distribuída, utilizando a biblioteca MPI (Message Passing Interface). A meta é investigar como diferentes estratégias de distribuição do trabalho entre processos afetam o tempo de execução, o balanceamento de carga e o custo de comunicação, comparando os resultados com a versão sequencial e com a solução OpenMP da Etapa 1.

A geração do fractal utiliza o algoritmo de tempo de escape (escape-time). Neste modelo, a imagem é tratada como um mapeamento do plano complexo, em que cada coordenada c (pixel) é submetida de forma independente à recorrência $z(n+1) = z(n)^2 + c$, partindo de $z(0) = 0$. O cálculo de um pixel termina quando $|z|^2 > 4$ ou quando o limite de iterações (MAX_ITER) é atingido, e o número de iterações realizadas é o valor do pixel.

Diferente do OpenMP, em que as threads compartilham a mesma memória, no MPI cada processo tem a sua própria memória. Por isso, a imagem precisa ser dividida explicitamente entre os processos e, no final, os resultados parciais precisam ser enviados por mensagens para um processo que monta a imagem completa. Todos os processos executam o mesmo programa e se diferenciam pelo seu identificador (rank), modelo conhecido como SPMD (Single Program, Multiple Data).

A divisão do trabalho pode ser estática, definida antes da execução (por exemplo, faixas de linhas contíguas ou linhas distribuídas de forma cíclica), ou dinâmica, em que um processo mestre entrega novas tarefas aos outros processos conforme eles terminam o que receberam (esquema mestre-trabalhador).

Embora o cálculo dos pixels não tenha dependência de dados, o problema de Mandelbrot apresenta um forte desbalanceamento no custo de cada pixel. Pixels localizados nas extremidades do plano escapam em poucas iterações, enquanto os pixels do interior do conjunto ou da sua fronteira (como o "vale dos cavalos-marinhos") chegam ao limite de iterações. Assim, dividir a imagem em partes com o mesmo número de pixels não garante a mesma quantidade de trabalho: os processos que recebem regiões fáceis terminam cedo e ficam ociosos, enquanto o tempo total é definido pelo processo mais sobrecarregado.

Em memória distribuída, o desempenho também depende da comunicação. O tempo total de um programa paralelo é a soma do tempo de computação, do tempo ocioso e do tempo de comunicação, e o tempo de comunicação é formado pela latência (preparo da mensagem) e pelo tempo de transmissão (largura de banda). Por isso, além de Speedup, Eficiência e Fator de Balanceamento de Carga, este relatório mede o tempo gasto para coletar a imagem e, no esquema mestre-trabalhador, o tempo de troca de mensagens.

## 2. Metodologia

A condução do benchmark foi dividida em etapas de validação e medição de desempenho. A versão sequencial de referência é a mesma da Etapa 1, adaptada para C a partir do código em Python fornecido pelo professor. Ela percorre a imagem com dois laços de repetição aninhados (linhas y e colunas x), calcula o número de iterações de cada pixel e armazena o resultado em um vetor unidimensional em ordem row-major. A versão MPI usa exatamente a mesma função de cálculo do pixel; a diferença está apenas na forma de dividir o trabalho entre os processos.

Na versão MPI, o processo de rank 0 lê os parâmetros de execução (resolução, região do plano, MAX_ITER, estratégia e tamanho do bloco) e os envia para os demais processos com a função MPI_Bcast. A estratégia de distribuição e o tamanho do bloco de linhas (chunk) são escolhidos no momento da execução pela variável de ambiente MPI_SCHEDULE (por exemplo, MPI_SCHEDULE=dynamic,8), da mesma forma que a variável OMP_SCHEDULE foi usada na Etapa 1. Isso permitiu executar os testes em lote sem recompilar o programa.

Foram implementadas três estratégias de distribuição. Em todas elas, a unidade de trabalho é a linha da imagem, o que mantém o acesso em ordem row-major e permite enviar os resultados como blocos contínuos de memória:

- **Block (faixas contíguas):** cada processo recebe uma faixa de linhas consecutivas, com aproximadamente $H/p$ linhas (H é a altura da imagem e p o número de processos). Quando a divisão não é exata, os primeiros processos recebem uma linha a mais. No final, a imagem é montada no rank 0 com a função MPI_Gatherv, que permite que cada processo envie uma quantidade diferente de dados.
- **Cyclic (distribuição cíclica):** as linhas são agrupadas em blocos de chunk linhas, que são distribuídos em rodízio entre os processos (o bloco 0 vai para o processo 0, o bloco 1 para o processo 1, e assim por diante). Foram testados chunks de 1, 8, 32 e 128 linhas. Como as linhas de cada processo não ficam juntas na imagem final, o rank 0 recebe os dados com MPI_Gatherv e depois coloca cada bloco na sua posição correta.
- **Dynamic (mestre-trabalhador):** o rank 0 atua apenas como mestre e não calcula pixels. Ele envia a cada trabalhador (MPI_Send) o número da primeira linha de uma tarefa de chunk linhas. Quando um trabalhador termina, envia as linhas calculadas de volta ao mestre, que guarda o resultado e envia uma nova tarefa ao mesmo trabalhador. Quando não há mais linhas, o mestre envia uma mensagem de término. O mestre usa MPI_Recv com MPI_ANY_SOURCE para atender primeiro quem terminar primeiro. Foram testados chunks de 1, 8, 32 e 128 linhas. Com p processos existem p − 1 trabalhadores; com apenas 1 processo, o programa executa a estratégia block.

Para a análise do balanceamento de carga, cada processo registra localmente a quantidade de pixels calculados, o total de iterações, o tempo gasto no cálculo e o tempo gasto com comunicação. Ao final, esses valores são reunidos no rank 0 com a

função MPI_Gather, onde são calculados os tempos mínimo, médio e máximo e o Fator de Balanceamento de Carga. Somente o rank 0 grava os arquivos de saída: a matriz de contagens em binário e a imagem em formato PPM.

Os testes de desempenho foram executados na Máquina 2 da Etapa 1, com um processador Intel Core i5-12450H (12ª geração, arquitetura híbrida) com 8 núcleos físicos operando a 2 GHz, sendo 4 núcleos de alta performance (P-Cores, com suporte a Hyper-Threading) e 4 núcleos de eficiência (E-Cores), totalizando 12 threads lógicas, e 20 GB de memória RAM. O sistema utilizado foi o Windows 11 25H2 com o Windows Subsystem for Linux (WSL 2.6.3.0, Ubuntu 24.04). Os programas foram compilados com GCC 13.3.0 (flags -O3 -Wall) e Open MPI 4.1.6, e executados com o comando mpirun -np N, com N igual a 1, 2, 4, 6, 8 e 12 processos, sem fixação explícita de processos em núcleos (binding).

Para a medição de tempo, todos os processos são sincronizados com MPI_Barrier antes do início do cronômetro, feito com a função MPI_Wtime. O tempo é medido no rank 0, desde o início da distribuição do trabalho até a imagem completa estar montada na memória do rank 0. Assim, o tempo de execução inclui o cálculo e a coleta da imagem, mas não inclui a escrita dos arquivos em disco, que foi medida separadamente. Cada configuração foi executada 5 vezes e foi utilizada a mediana dos tempos.

O tempo de comunicação foi medido da seguinte forma: nas estratégias block e cyclic, é o tempo da coleta com MPI_Gatherv (no cyclic, incluindo a reorganização das linhas), medido depois de uma barreira para não incluir a espera pelos processos mais lentos. Na estratégia dynamic, é o tempo que cada trabalhador gasta esperando uma nova tarefa e enviando seus resultados, calculado como a média entre os trabalhadores. A coluna "Mestre" das tabelas mostra o tempo total em que o mestre ficou ativo distribuindo tarefas.

Os casos de teste seguem o enunciado: o caso padrão (região $Re \in [-2{,}0;\ 1{,}0]$ e $Im \in [-1{,}5;\ 1{,}5]$, resolução 4096x4096 e MAX_ITER = 1000), o caso de desbalanceamento (zoom no "vale dos cavalos-marinhos", com centro em (−0,743643887; 0,131825904), largura de $3{,}0\times10^{-3}$ e MAX_ITER = 5000) e a escalabilidade fraca, em que a resolução cresce junto com o número de processos ($N = 4096\cdot\sqrt{p}$, ou seja, 4096, 5793, 8192, 10033, 11585 e 14189 para 1, 2, 4, 6, 8 e 12 processos), mantendo aproximadamente o mesmo número de pixels por processo.

A avaliação de desempenho baseou-se nas seguintes métricas:

**Tempo de Execução:** tempo total gasto para calcular e montar a imagem no rank 0.

**Speedup (S):** mede o ganho de velocidade, calculado pela razão entre o tempo sequencial de referência Tseq e o tempo da execução paralela com p processos Tpar.

$$S = \frac{T_{seq}}{T_{par}}$$

**Eficiência (E):** expressa o quanto os processos foram aproveitados, obtida pela divisão do Speedup pelo número de processos utilizados. Na estratégia dynamic, o mestre também é contado, mesmo sem calcular pixels.

$$E = \frac{S}{p} \times 100\%$$

**Fator de Balanceamento de Carga (FBC):** O FBC mede a diferença entre o processo que mais trabalhou e o que menos trabalhou. Tmax e Tmin são o maior e o menor tempo de computação entre os processos que calculam pixels (o mestre da estratégia dynamic não entra nesse cálculo). Apenas o tempo de computação foi considerado, sem o tempo de comunicação. Valores próximos de 0 indicam boa distribuição de carga e valores próximos de 1 indicam forte desbalanceamento.

$$FBC = \frac{T_{max} - T_{min}}{T_{max}}$$

**Escalabilidade:** avaliada de forma forte (resolução fixa e número de processos variando) e fraca (resolução crescendo junto com o número de processos), e relacionada às leis de Amdahl e Gustafson na Seção 4.

# 3. Resultados

## 3.1 Teste de Corretude

A corretude da implementação MPI foi verificada por meio da comparação direta entre o arquivo binário com a matriz de contagens de iteração gerada pela versão sequencial e o arquivo gerado pela versão MPI. Para isso, foi utilizado o comando *cmp mandelbrot_matriz.bin mandelbrot_mpi_matriz.bin && echo "IGUAIS" || echo "DIFERENTES"*. O utilitário cmp compara os dois arquivos byte a byte. O resultado obtido foi "IGUAIS", indicando que não houve nenhuma diferença entre as matrizes. Dessa forma, a versão MPI preservou exatamente o resultado da versão sequencial de referência, sem precisar da tolerância de até 0,01% dos pixels prevista no enunciado. A mesma comparação também é feita automaticamente pelo script de benchmark para as três estratégias, antes das medições de tempo.

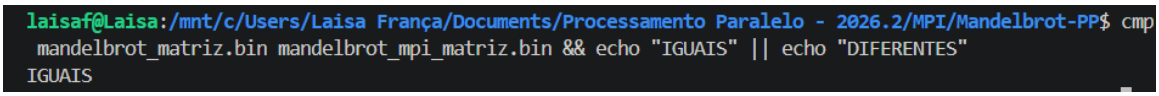



*Figura 1 – Resultado do teste de corretude entre as versões sequencial e MPI.*

## 3.2 Caso Base - Tempo de execução serial

O tempo de execução da versão sequencial na Máquina 2 foi de **6,989 s** para o caso padrão (4096x4096, MAX_ITER = 1000) e de **3,037s** para o caso de desbalanceamento (4096x4096, MAX_ITER = 5000). Esses tempos são as referências para o cálculo do Speedup nas próximas seções.

A escrita em disco dos arquivos de saída (matriz binária e imagem PPM) levou 56,08 s e foi medida separadamente, fora do tempo de execução. Quase todo esse tempo vem da imagem PPM, que é gravada em formato de texto.

### 3.3 Casos de Teste Estratégia/Chunk

***Tabela 1:*** Resultados da avaliação das estratégias de distribuição e tamanhos de chunk, utilizando resolução 4096x4096, MAX_ITER = 1000 e 12 processos.

| Estratégia | Chunk | Tempo (s) | Comun. (s) | Mestre (s) | Speedup | Efic. (%) | FBC |
|---|---|---|---|---|---|---|---|
| block | padrão | 3,031 | 0,050 | 0,000 | 2,31 | 19,21 | 0,997 |
| cyclic | 1 | 1,312 | 0,115 | 0,000 | 5,33 | 44,40 | 0,232 |
| cyclic | 8 | 1,239 | 0,080 | 0,000 | 5,64 | 47,01 | 0,226 |
| cyclic | 32 | 1,438 | 0,113 | 0,000 | 4,86 | 40,51 | 0,317 |
| cyclic | 128 | 1,525 | 0,073 | 0,000 | 4,58 | 38,19 | 0,452 |
| dynamic | 1 | 1,164 | 0,096 | 1,164 | 6,00 | 50,02 | 0,020 |
| dynamic | 8 | 1,159 | 0,048 | 1,159 | 6,03 | 50,25 | 0,023 |
| dynamic | 32 | 1,142 | 0,029 | 1,142 | 6,12 | 50,99 | 0,015 |
| dynamic | 128 | 1,315 | 0,008 | 1,315 | 5,31 | 44,28 | 0,323 |

A estratégia dynamic obteve os menores tempos, com o melhor resultado em dynamic,32 (1,142s e Speedup de 6,12). A estratégia block teve o pior resultado (3,031s), com FBC de 0,997.

#### 3.3.1 Representação Gráfica

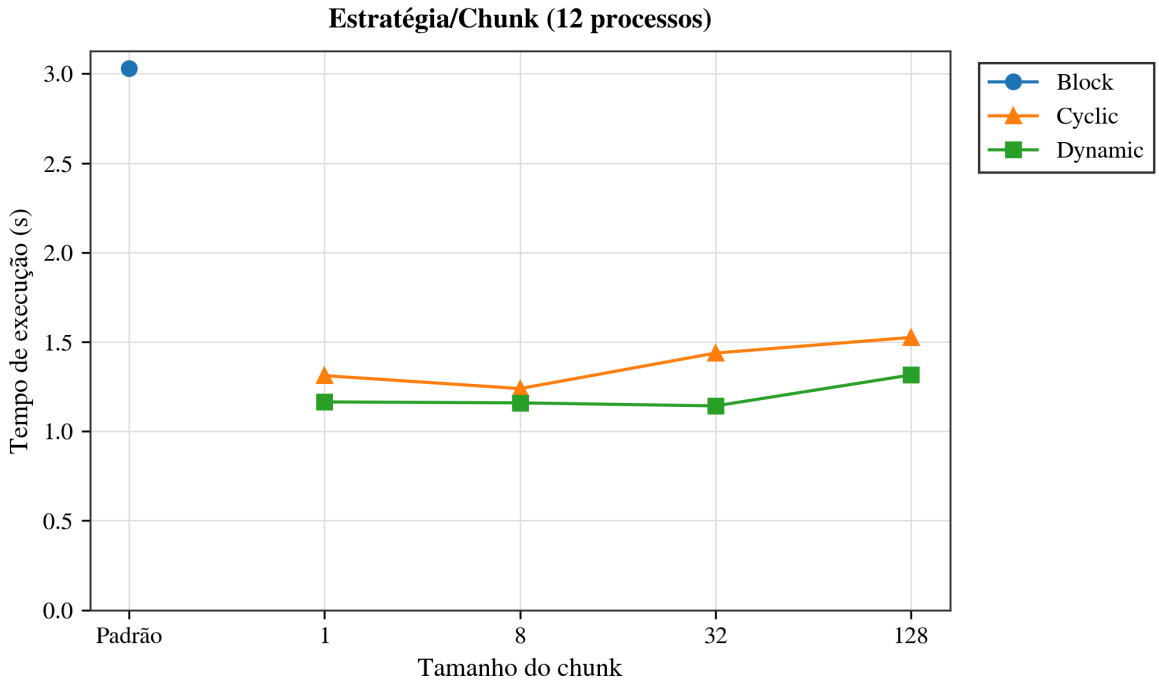



*Figura 2 – Comparação do Tempo de Execução das estratégias de distribuição e tamanhos de chunk com 12 processos.*

## *Speedup e Eficiência*

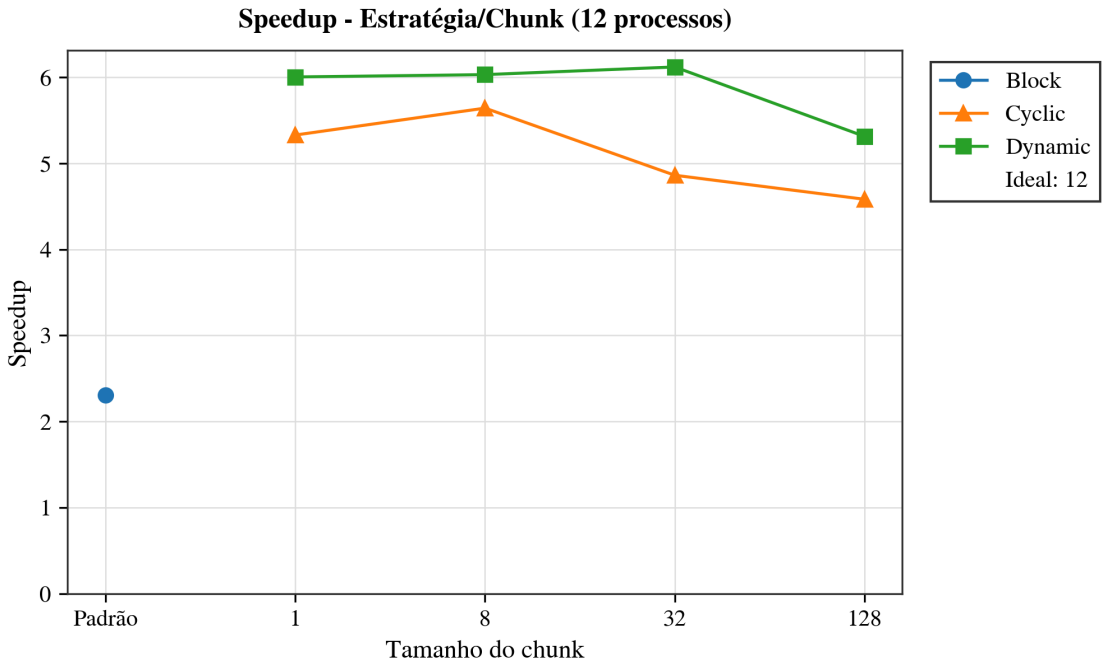



*Figura 3 – Speedup obtido para as diferentes estratégias e chunks. O Speedup foi calculado com base no tempo sequencial de referência de 6,989s.*

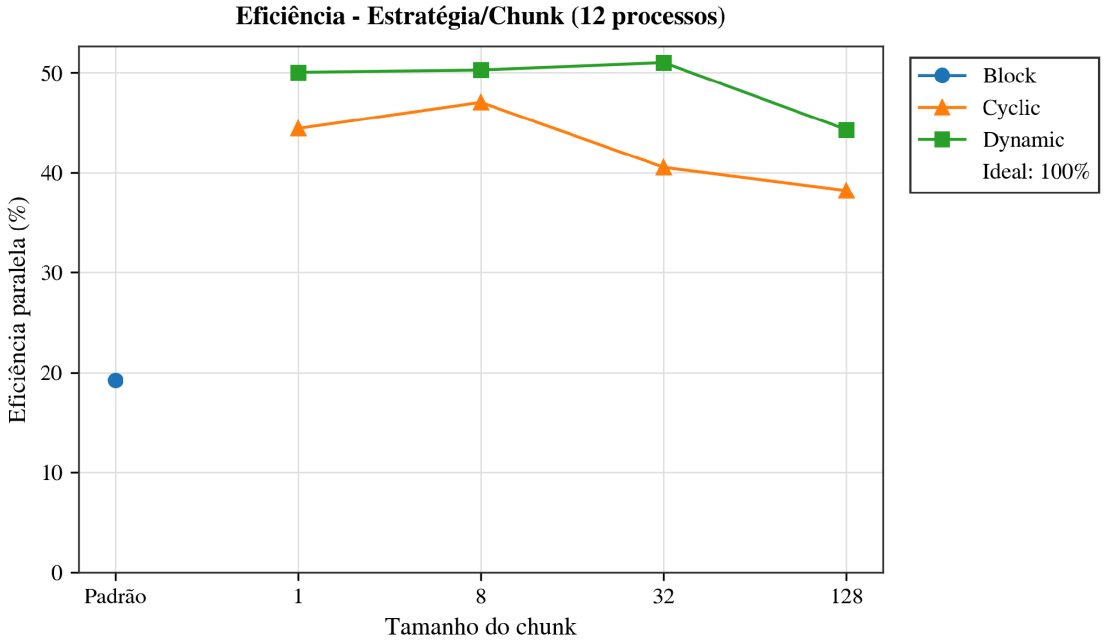



*Figura 4 – Eficiência Paralela obtida para as diferentes estratégias e chunks com 12 processos.*

### *Carga mínima, média e máxima*

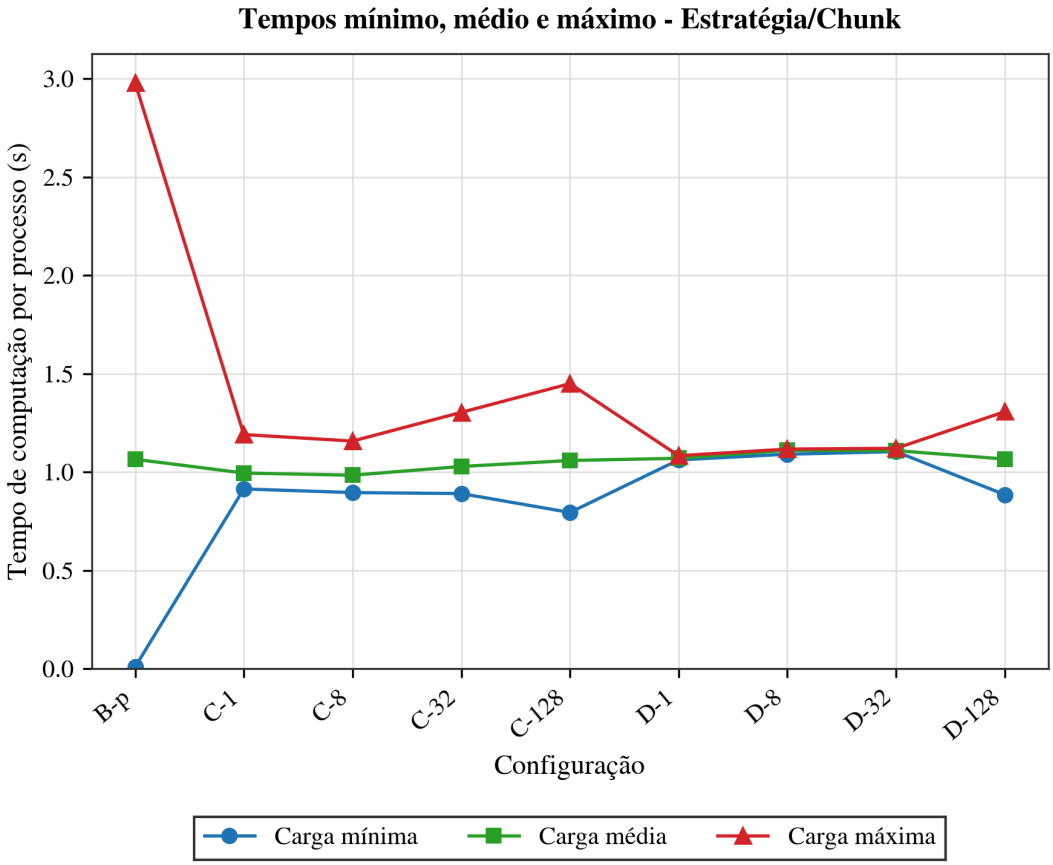



*Figura 5 – Tempos de computação mínimo, médio e máximo entre os processos. Quanto mais próximas estiverem as três curvas, melhor é o balanceamento de carga.*

## *Fator de Balanceamento de Carga*

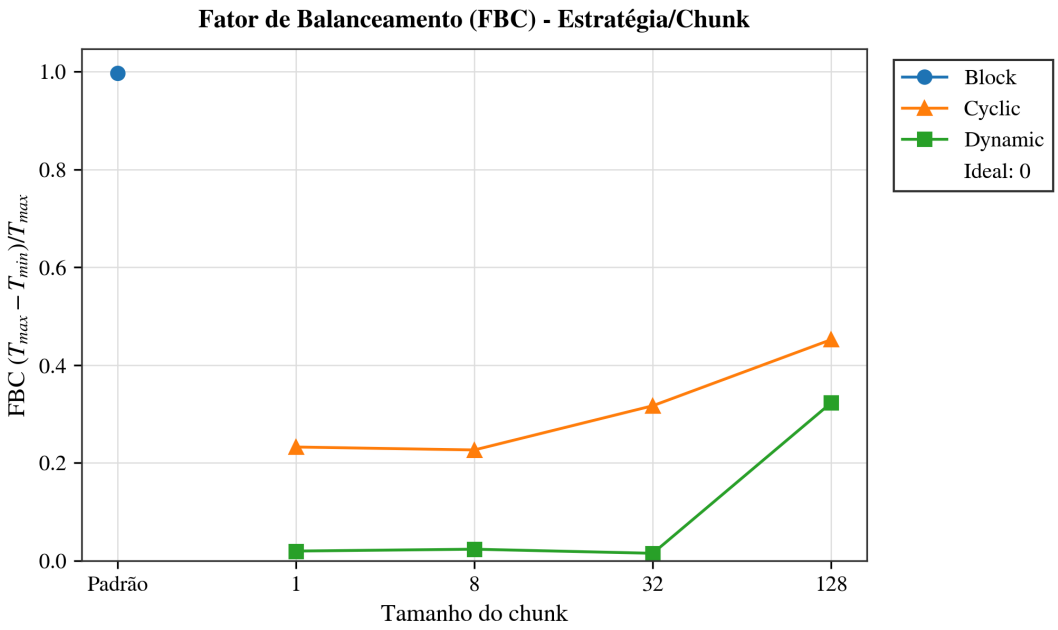



*Figura 6 – Fator de Balanceamento de Carga obtido para as diferentes estratégias e chunks. Valores mais próximos de 0 indicam melhor distribuição de carga.*

### *Custo de Comunicação*

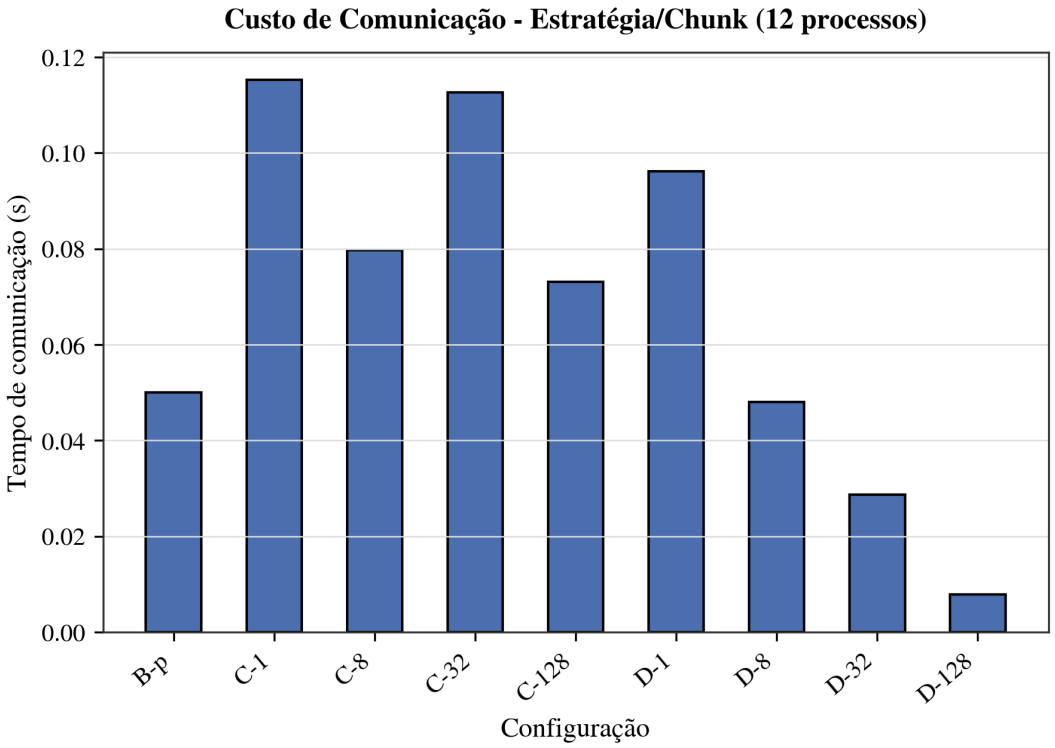



*Figura 7 – Tempo de comunicação para as diferentes estratégias e chunks com 12 processos.*

## 3.4 Casos de Estresse

### 3.4.1 Escalabilidade Forte (Casos de Teste)

***Tabela 2:*** Resultados de escalabilidade forte para as estratégias block, cyclic,1 e dynamic,1, com resolução fixa de 4096x4096 e MAX_ITER = 1000.

| Estratégia | Processos | Tempo (s) | Comun. (s) | Mestre (s) | Speedup | Efic. (%) | FBC |
|---|---|---|---|---|---|---|---|
| block | 1 | 6.935980 | 0.040961 | 0.000000 | 1.008 | 100.76 | 0.000000 |
| block | 2 | 3.610548 | 0.042804 | 0.000000 | 1.936 | 96.79 | 0.001799 |
| block | 4 | 4.637556 | 0.037872 | 0.000000 | 1.507 | 37.68 | 0.976051 |
| block | 6 | 4.468208 | 0.047674 | 0.000000 | 1.564 | 26.07 | 0.994422 |
| block | 8 | 3.745733 | 0.044610 | 0.000000 | 1.866 | 23.32 | 0.995368 |
| block | 12 | 3.691861 | 0.053378 | 0.000000 | 1.893 | 15.78 | 0.996464 |
| cyclic,1 | 1 | 6.907587 | 0.084666 | 0.000000 | 1.012 | 101.18 | 0.000000 |
| cyclic,1 | 2 | 3.795236 | 0.082678 | 0.000000 | 1.842 | 92.08 | 0.000232 |
| cyclic,1 | 4 | 2.067935 | 0.076005 | 0.000000 | 3.380 | 84.49 | 0.006671 |
| cyclic,1 | 6 | 1.692399 | 0.103514 | 0.000000 | 4.130 | 68.83 | 0.067286 |
| cyclic,1 | 8 | 1.403162 | 0.084564 | 0.000000 | 4.981 | 62.26 | 0.037021 |
| cyclic,1 | 12 | 1.260989 | 0.103627 | 0.000000 | 5.542 | 46.19 | 0.226196 |
| dynamic,1 | 1 | 6.919087 | 0.040601 | 0.000000 | 1.010 | 101.01 | 0.000000 |
| dynamic,1 | 2 | 8.410392 | 0.084749 | 8.410389 | 0.831 | 41.55 | 0.000000 |
| dynamic,1 | 4 | 3.113538 | 0.028229 | 3.113535 | 2.245 | 56.12 | 0.001087 |
| dynamic,1 | 6 | 2.000504 | 0.034195 | 2.000500 | 3.494 | 58.23 | 0.004087 |
| dynamic,1 | 8 | 1.496518 | 0.034394 | 1.496505 | 4.670 | 58.38 | 0.006138 |
| dynamic,1 | 12 | 1.124384 | 0.035843 | 1.124374 | 6.216 | 51.80 | 0.011247 |

As estratégias cyclic,1 e dynamic,1 reduziram o tempo à medida que o número de processos aumentou, e a dynamic,1 terminou com o menor tempo com 12 processos (1,124s e Speedup de 6,22). A estratégia block deixou de melhorar a partir de 4 processos. Com 2 processos, a dynamic,1 ficou mais lenta que a versão sequencial (Speedup de 0,83), pois nesse caso existe apenas um trabalhador.

#### 3.4.1.1 Representação Gráfica

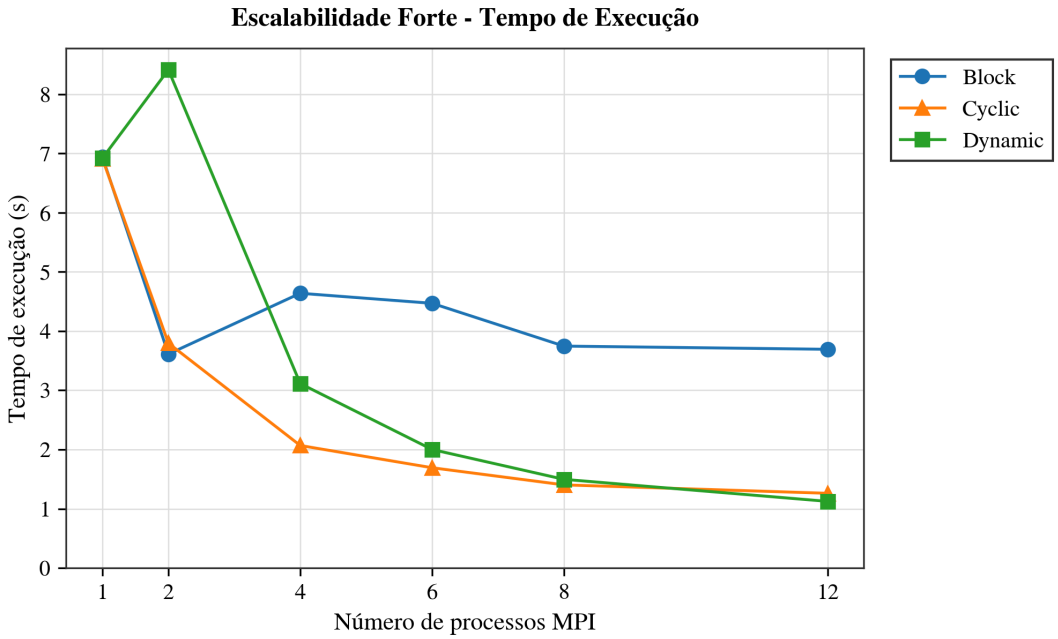



*Figura 8 – Tempo de Execução na análise de escalabilidade forte para as estratégias block, cyclic,1 e dynamic,1.*

### *Speedup e Eficiência*

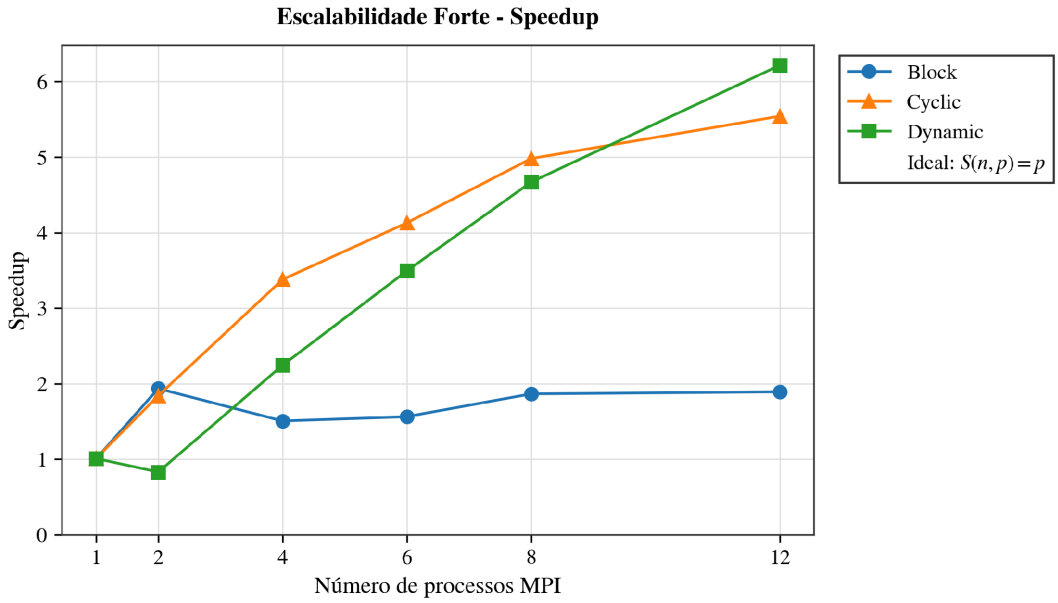



*Figura 9 – Speedup obtido na análise de escalabilidade forte. O Speedup foi calculado com base no tempo sequencial de referência de 6,989s.*

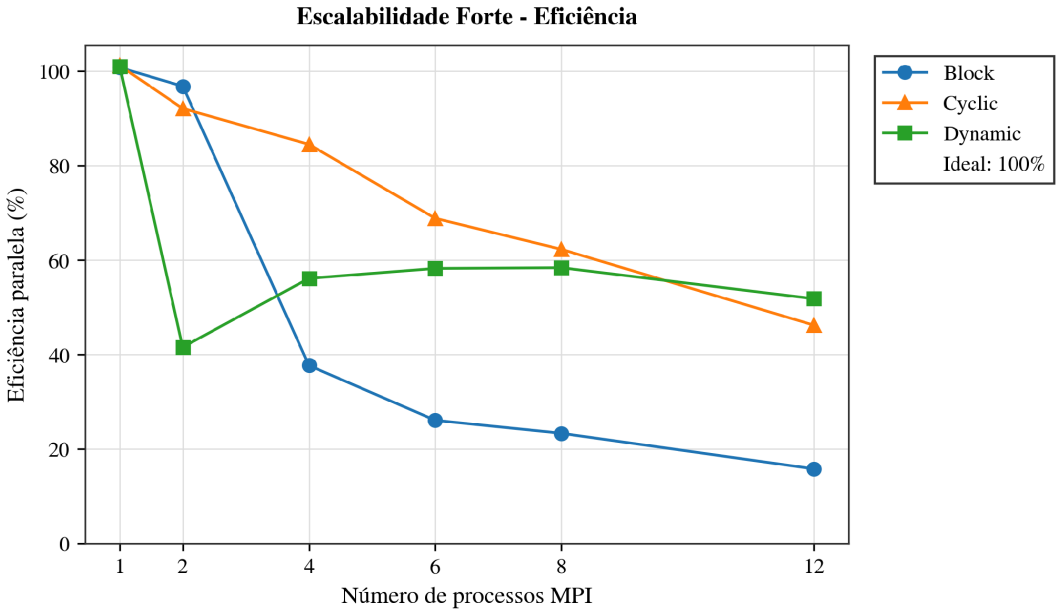



*Figura 10 – Eficiência paralela obtida na análise de escalabilidade forte.*

## *Fator de Balanceamento de Carga*

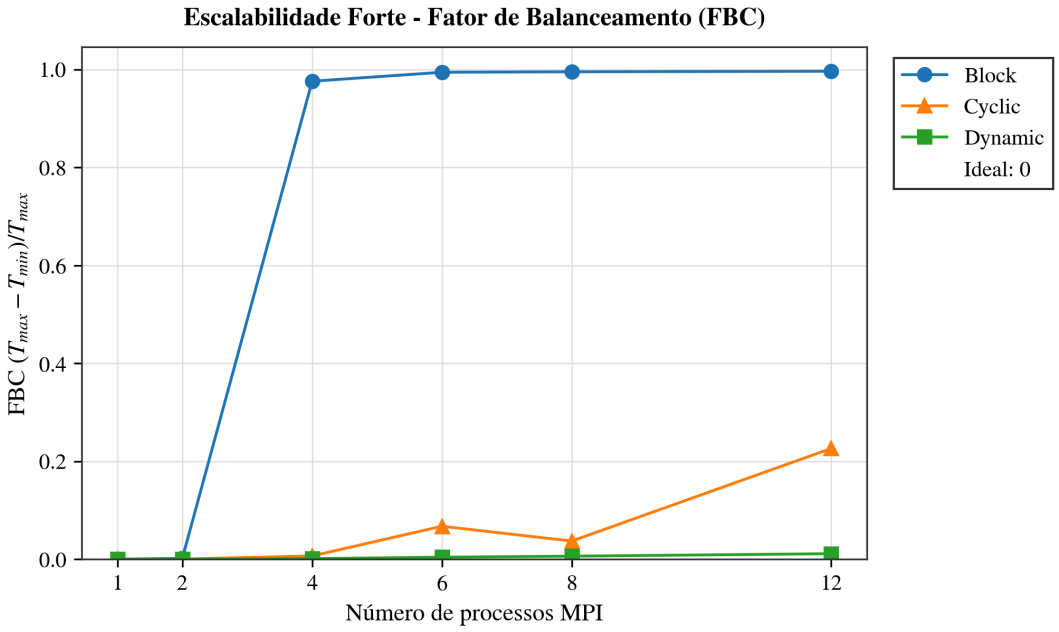



*Figura 11 – Fator de Balanceamento de Carga obtido na análise de escalabilidade forte.*

### *Custo de Comunicação*

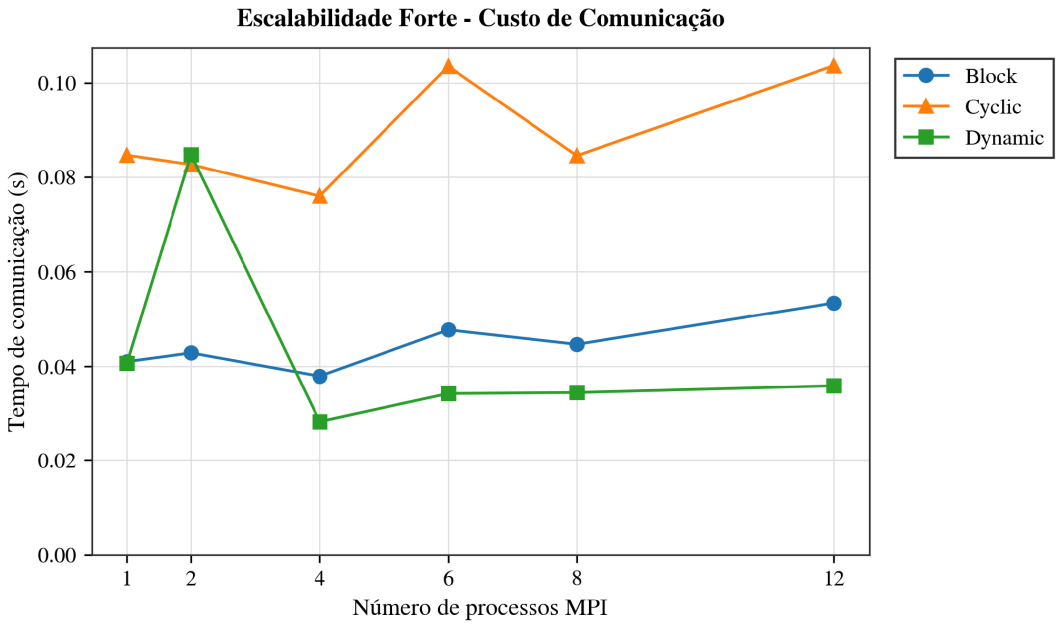



*Figura 12 – Tempo de comunicação na análise de escalabilidade forte.*

#### 3.4.2 Desbalanceamento (Casos de Teste)

***Tabela 3:*** Resultados dos testes de desbalanceamento utilizando resolução 4096x4096, MAX_ITER = 5000 e 12 processos.

| Estratégia | Chunk | Tempo (s) | Comun. (s) | Mestre (s) | Speedup | Efic. (%) | FBC |
|---|---|---|---|---|---|---|---|
| block | padrão | 0.777664 | 0.062099 | 0.000000 | 3.905 | 32.54 | 0.718603 |
| cyclic | 1 | 0.599543 | 0.096206 | 0.000000 | 5.065 | 42.21 | 0.214914 |
| cyclic | 8 | 0.618538 | 0.076408 | 0.000000 | 4.909 | 40.91 | 0.259881 |
| cyclic | 32 | 0.666334 | 0.095595 | 0.000000 | 4.557 | 37.98 | 0.334407 |
| cyclic | 128 | 0.711265 | 0.104938 | 0.000000 | 4.269 | 35.58 | 0.511430 |
| dynamic | 1 | 0.534585 | 0.047816 | 0.534575 | 5.680 | 47.34 | 0.056556 |
| dynamic | 8 | 0.522106 | 0.014747 | 0.522099 | 5.816 | 48.47 | 0.044353 |
| dynamic | 32 | 0.542992 | 0.012085 | 0.542944 | 5.592 | 46.60 | 0.114030 |
| dynamic | 128 | 0.661809 | 0.006185 | 0.661618 | 4.588 | 38.24 | 0.391880 |

A configuração dynamic,8 obteve o menor tempo (0,522s, Speedup de 5,82 e FBC de 0,044).

### 3.4.2.1 Representação Gráfica

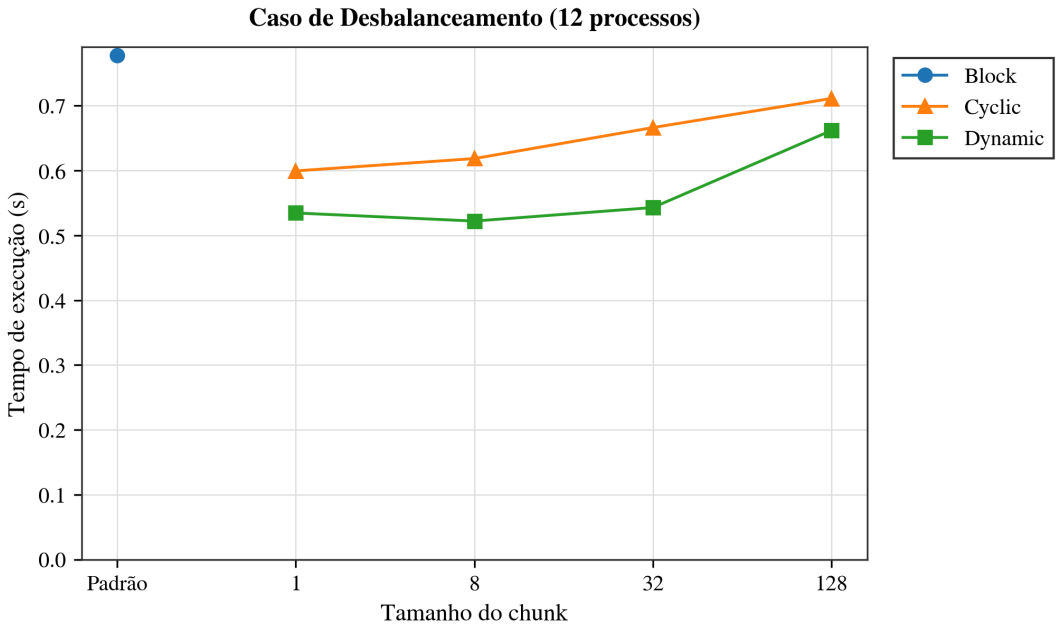



*Figura 13 – Tempo de Execução das estratégias de distribuição no caso de desbalanceamento.*

## *Speedup e Eficiência*

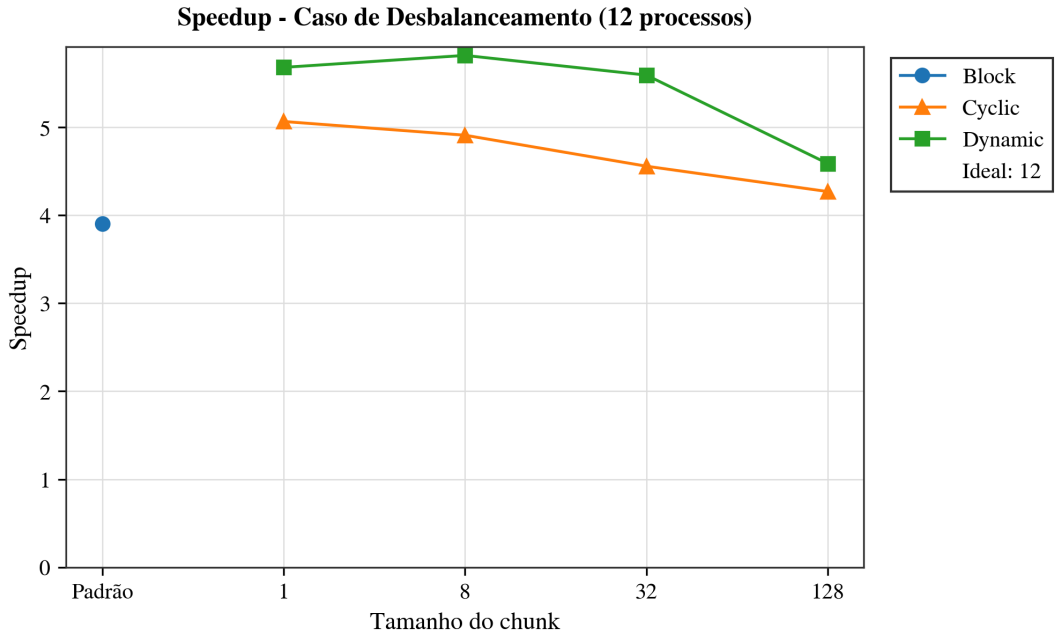



*Figura 14 – Speedup obtido no caso de desbalanceamento. O Speedup foi calculado com base no tempo sequencial de referência de 3,037s.*

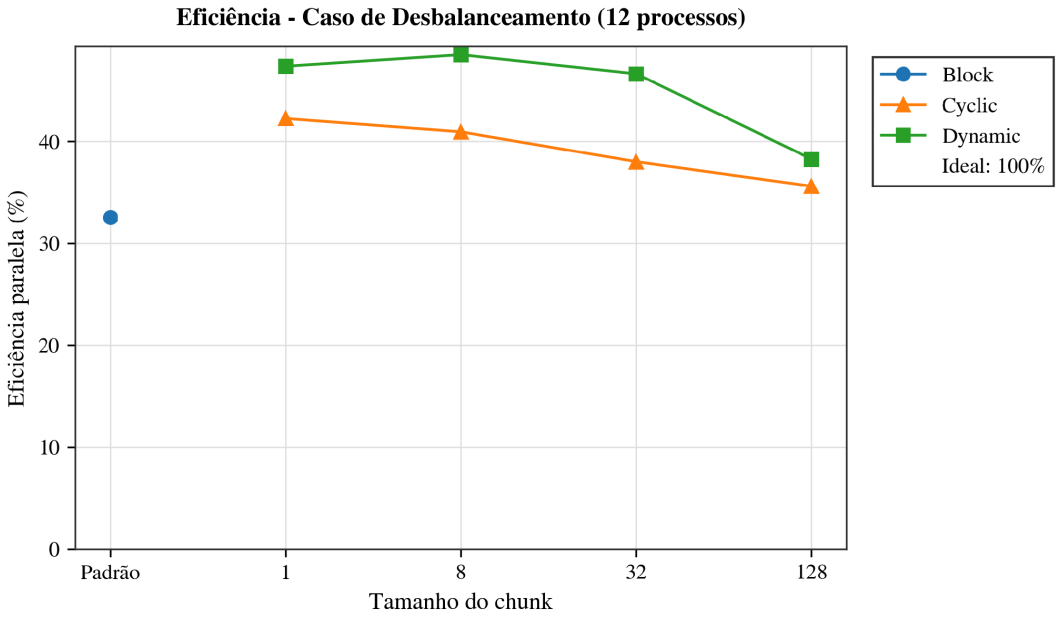



*Figura 15 – Eficiência obtida pelas diferentes estratégias no caso de desbalanceamento.*

### *Carga mínima, média e máxima*

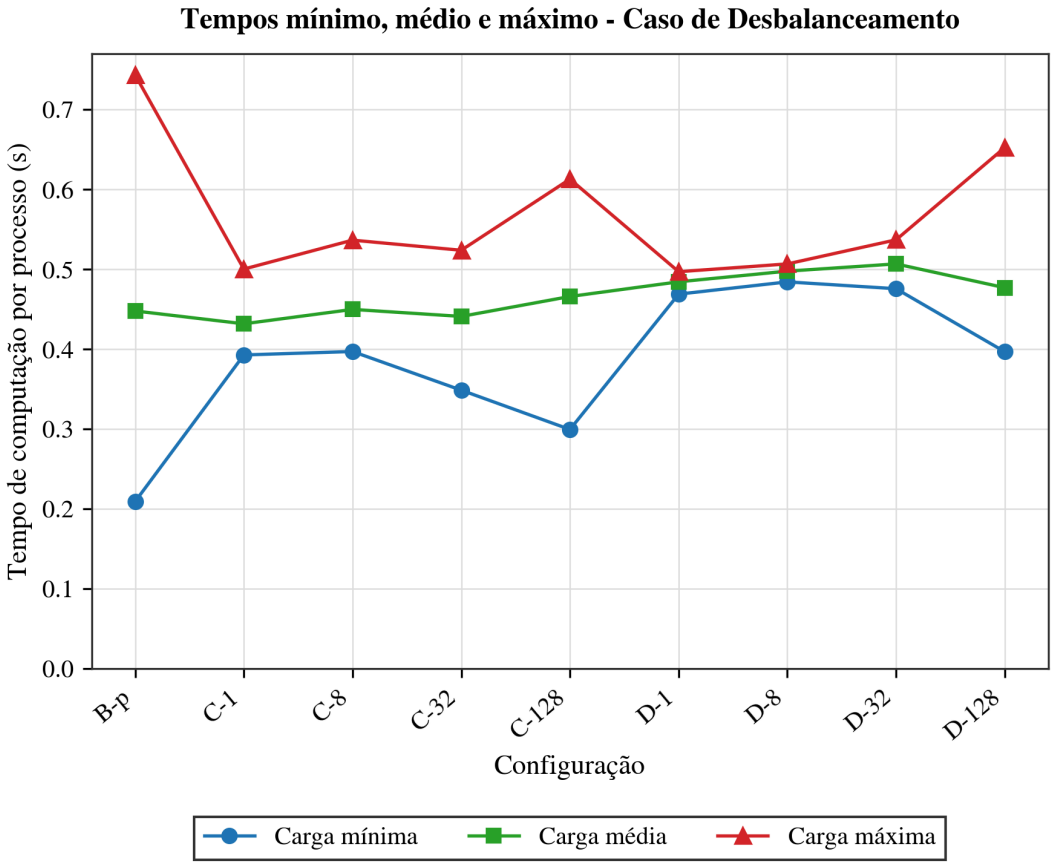



*Figura 16 – Tempos de computação mínimo, médio e máximo entre os processos no caso de desbalanceamento. Quanto mais próximas estiverem as três curvas, melhor é o balanceamento de carga.*

### *Fator de Balanceamento de Carga*

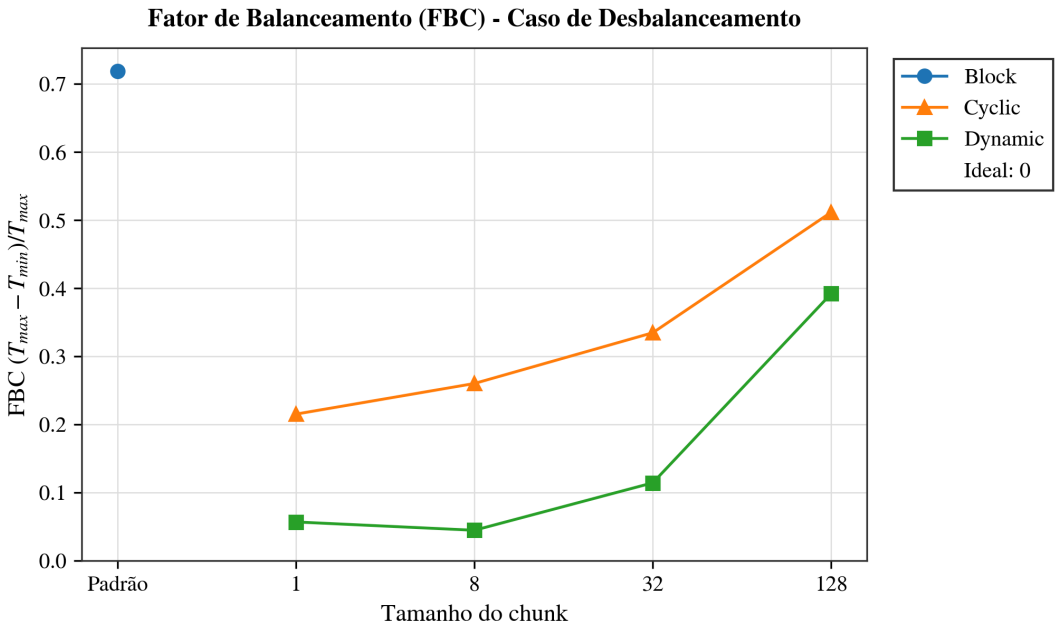



*Figura 17 – Fator de Balanceamento de Carga no caso de desbalanceamento. Valores mais próximos de 0 indicam melhor distribuição de carga.*

### *Custo de Comunicação*

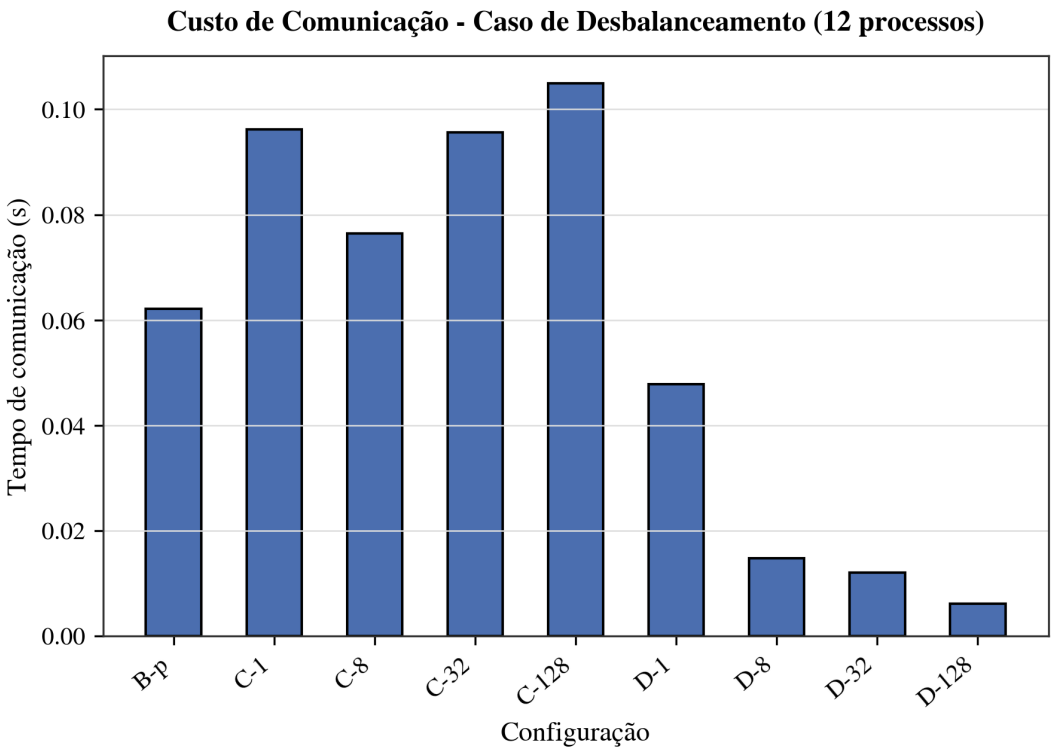



*Figura 18 – Tempo de comunicação no caso de desbalanceamento.*

#### 3.4.3 Escalabilidade Fraca (Casos de Teste)

***Tabela 4:*** Resultados de escalabilidade fraca utilizando a configuração dynamic,1, MAX_ITER = 1000 e variando a resolução junto com o número de processos (1, 2, 4, 6, 8 e 12). O Speedup de cada linha usa o tempo sequencial da mesma resolução.

| Resolução | Processos | Tempo (s) | Comun. (s) | Serial ref. (s) | Speedup | Efic. (%) | FBC |
|---|---|---|---|---|---|---|---|
| 4096×4096 | 1 | 6.846121 | 0.038471 | 6.989003 | 1.021 | 102.09 | 0.000000 |
| 5793×5793 | 2 | 16.467543 | 0.159986 | 13.839668 | 0.840 | 42.02 | 0.000000 |
| 8192×8192 | 4 | 12.204588 | 0.111146 | 28.727478 | 2.354 | 58.85 | 0.000701 |
| 10033×10033 | 6 | 12.333086 | 0.142399 | 44.032495 | 3.570 | 59.50 | 0.001846 |
| 11585×11585 | 8 | 12.350634 | 0.207781 | 55.728086 | 4.512 | 56.40 | 0.001315 |
| 14189×14189 | 12 | 13.805562 | 0.392543 | 82.337333 | 5.964 | 49.70 | 0.005630 |

Entre 4 e 8 processos, o tempo ficou praticamente constante, entre 12,2s e 12,4s, e subiu para 13,8s com 12 processos. O tempo de comunicação cresceu de 0,038s para 0,393s, pois a imagem coletada fica maior a cada resolução (cerca de 805 MB na resolução 14189x14189).

#### 3.4.3.1 Representação Gráfica

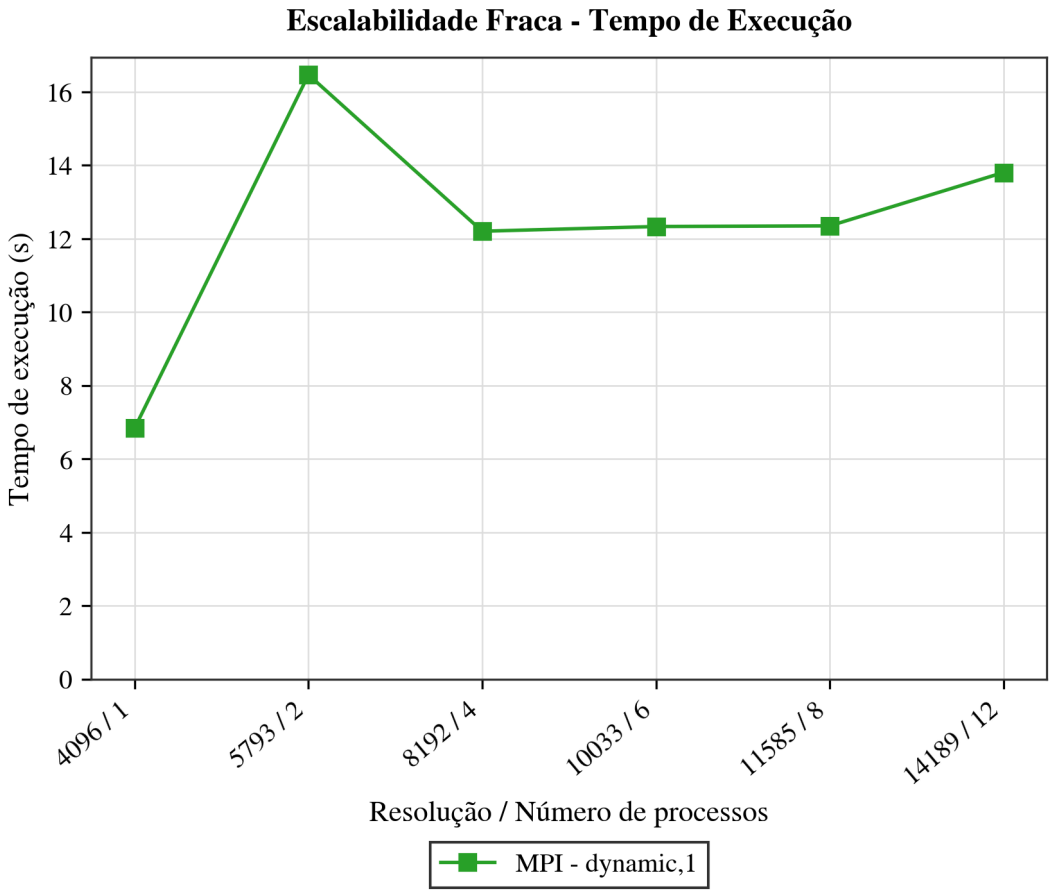



*Figura 19 – Tempo de execução obtido nos experimentos de escalabilidade fraca utilizando a configuração dynamic,1. O tamanho do problema foi aumentado junto com o número de processos.*

### *Speedup e Eficiência*

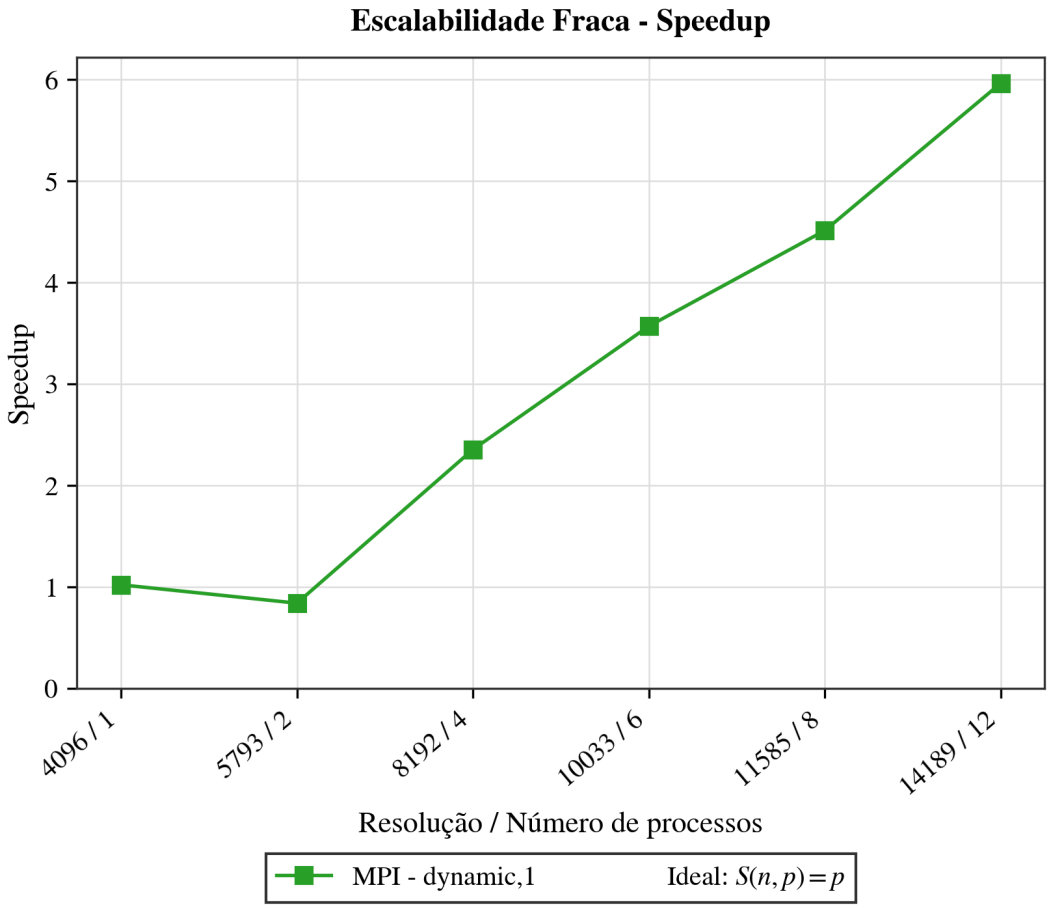



*Figura 20 – Speedup obtido nos experimentos de escalabilidade fraca. Cada ponto foi calculado a partir do tempo sequencial da mesma resolução.*

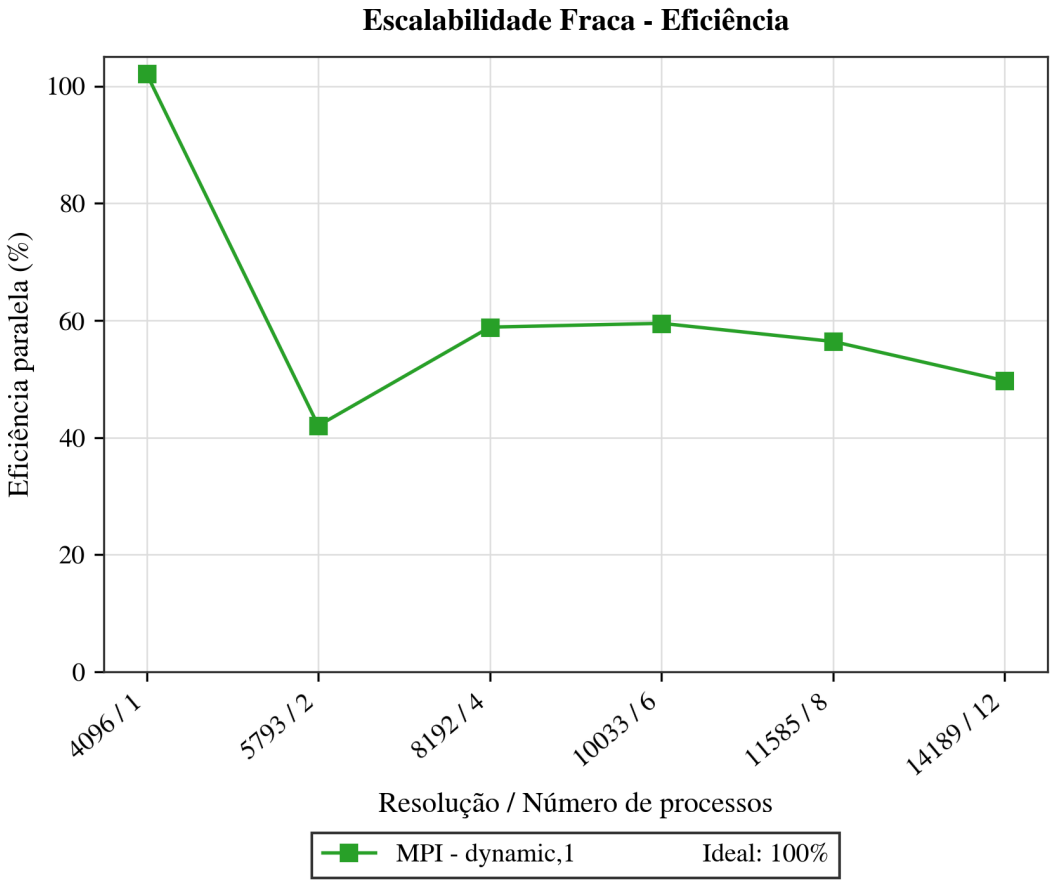



*Figura 21 – Eficiência paralela obtida nos experimentos de escalabilidade fraca.*

***Fator de Balanceamento de Carga***

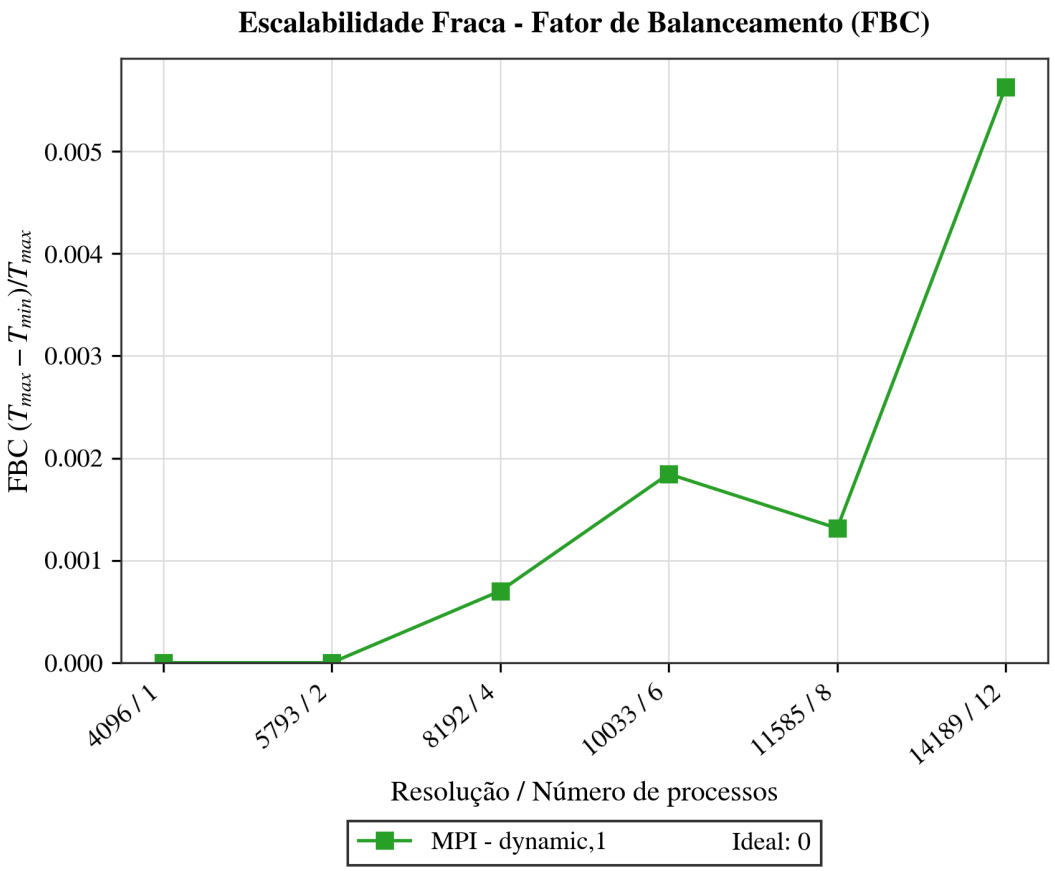



*Figura 22 – Fator de Balanceamento de Carga obtido nos experimentos de escalabilidade fraca.*

### 3.4.4 Comparação com a solução OpenMP

***Tabela 5:*** Comparação de desempenho entre OpenMP (dynamic,1) e MPI (cyclic,1 e dynamic,1), com resolução 4096x4096 e MAX_ITER = 1000. Unidades representam threads no OpenMP e processos no MPI.

| | Tempo (s) | | | Speedup | | Eficiência (%) | |
|---|---|---|---|---|---|---|---|
| **Unidades** | **OpenMP** | **MPI cyclic** | **MPI dynamic** | **OpenMP** | **MPI dynamic** | **OpenMP** | **MPI dynamic** |
| 1 | 6.901151 | 6.907587 | 6.919087 | 0.973 | 1.010 | 97.33 | 101.01 |
| 2 | 3.575144 | 3.795236 | 8.410392 | 1.879 | 0.831 | 93.93 | 41.55 |
| 4 | 2.129895 | 2.067935 | 3.113538 | 3.153 | 2.245 | 78.84 | 56.12 |
| 6 | 1.663652 | 1.692399 | 2.000504 | 4.037 | 3.494 | 67.29 | 58.23 |
| 8 | 1.301849 | 1.403162 | 1.496518 | 5.159 | 4.670 | 64.49 | 58.38 |
| 12 | 1.035296 | 1.260989 | 1.124384 | 6.488 | 6.216 | 54.06 | 51.80 |

***Tabela 6:*** Resultado OpenMP com cada schedule e chunk, 12 threads.

| Schedule | Chunk | Tempo (s) | Speedup | Efic. (%) | Fator ($T_{max}/T_{médio}$) |
|---|---|---|---|---|---|
| static | padrão | 1.894891 | 3.545 | 29.54 | 2.859080 |
| static | 1 | 1.320089 | 5.088 | 42.40 | 1.237495 |
| static | 8 | 1.173705 | 5.723 | 47.69 | 1.264416 |
| static | 32 | 1.135230 | 5.916 | 49.30 | 1.209089 |
| static | 128 | 1.336138 | 5.027 | 41.89 | 1.319824 |
| dynamic | 1 | 1.093314 | 6.143 | 51.19 | 1.006141 |
| dynamic | 8 | 0.992940 | 6.764 | 56.37 | 1.002424 |
| dynamic | 32 | 0.945864 | 7.101 | 59.17 | 1.018969 |
| dynamic | 128 | 1.091897 | 6.151 | 51.26 | 1.197818 |
| guided | 1 | 1.324100 | 5.073 | 42.27 | 1.573142 |
| guided | 8 | 1.399706 | 4.799 | 39.99 | 1.572533 |
| guided | 32 | 1.438445 | 4.669 | 38.91 | 1.601484 |
| guided | 128 | 1.435737 | 4.678 | 38.98 | 1.625614 |

Os dados do OpenMP foram coletados novamente na mesma máquina e no mesmo período dos testes MPI, e o seu Speedup foi calculado com o tempo sequencial medido na bateria do OpenMP (6,717 s). Em todas as quantidades de unidades, o OpenMP teve tempo menor que o MPI dynamic,1. Com 12 unidades, o OpenMP levou 1,035s e o MPI dynamic,1 levou 1,124s. Considerando a melhor configuração de cada um com 12 unidades, o OpenMP chegou a 0,946s (dynamic,32) e o MPI a 1,142s (dynamic,32). No caso de desbalanceamento, o melhor resultado do OpenMP foi 0,492s (dynamic,1) e o do MPI foi 0,522s (dynamic,8).

# 4. Discussão

## 4.1 Análise da Corretude da Implementação

Como o cálculo do Conjunto de Mandelbrot não tem dependência de dados entre os pixels, cada pixel é calculado por um único processo, com a mesma função e a mesma ordem de operações da versão sequencial. Por isso, o resultado foi idêntico byte a byte. Em memória distribuída não existe risco de condição de corrida na escrita, porque cada processo escreve apenas na sua própria memória. O cuidado principal está na montagem da imagem, já que cada linha precisa voltar para a posição correta. Com 12 processos, as 4096 linhas não se dividem igualmente, então a igualdade dos resultados também confirma que as linhas que sobram na divisão foram tratadas corretamente.

## 4.2 Comparação entre a Versão Sequencial e a Versão Paralela em MPI

A versão sequencial levou 6,989s no caso padrão. Com 12 processos, a melhor configuração MPI (dynamic,32) reduziu esse tempo para 1,142s, um Speedup de 6,12 *(Tabela 1 e Figura 3)*. No caso de desbalanceamento, o tempo caiu de 3,037s para 0,522s com dynamic,8, atingindo um Speedup de 5,82 *(Tabela 3 e Figura 14)*. Mesmo a pior estratégia, block, foi cerca de duas vezes mais rápida que a versão sequencial com 12 processos, mas aproveitou mal os processos, com eficiência entre 15,8% e 19,2% *(Figura 4 e Tabela 2)*.

Seguindo a classificação, quase todos os resultados ficaram na faixa sublinear ($1 < S < p$), que é o comportamento esperado na prática, como mostra a Figura 9. A exceção foi a estratégia dynamic com 2 processos, que teve slowdown ($S < 1$), pois um dos dois processos é o mestre e apenas o outro calcula os pixels.

## 4.3 Comparação entre as Estratégias de Distribuição

**Block:** É a estratégia de implementação mais simples e a que apresenta o menor custo de coleta da imagem (um único MPI_Gatherv sem necessidade de reordenação, levando entre 0,038s e 0,053s na Tabela 2), mas não se adapta à geometria do fractal. Como a região mais custosa do Conjunto de Mandelbrot se concentra na faixa central da imagem, os processos responsáveis pelas bordas terminam rapidamente, enquanto os processos centrais concentram quase todo o trabalho. Na Figura 5, observa-se essa distância entre a curva de carga mínima (próxima de 0 s) e a de carga máxima (próxima de 3,0s), o que resultou no pior tempo do caso padrão com 12 processos (3,031 s, Speedup de 2,31 e FBC de 0,997 na Tabela 1). Na escalabilidade forte (Tabela 2), a estratégia block funcionou bem apenas com 2 processos (3,611 s e FBC de 0,002), porque a imagem é dividida em duas metades simétricas; a partir de 4 processos, o FBC ultrapassou 0,97 e o tempo de execução deixou de cair. No caso de desbalanceamento (Tabela 3 e Figura 16), a block também teve o pior desempenho (0,778s e FBC de 0,719), mantendo uma grande distância entre as curvas de carga mínima e máxima.

**Cyclic:** Distribuir as linhas de forma intercalada espalha as regiões mais custosas do fractal entre todos os processos antes da execução. Com 12 processos no caso padrão (Tabela 1), isso reduziu o tempo de execução de 3,031s (block) para 1,239s (cyclic com chunk 8, Speedup de 5,64), aproximando as curvas de carga mínima, média e máxima (Figura 5). Ainda assim, como a distribuição permanece estática, o aumento do tamanho do chunk volta a concentrar faixas pesadas em poucos processos: o FBC subiu de 0,226 (chunk 8) para 0,452 (chunk 128) no caso padrão (Tabela 1 e Figura 6), e de 0,215 (chunk 1) para 0,511 (chunk 128) no caso de desbalanceamento (Tabela 3 e Figura 17). Além disso, a cyclic exige que o rank 0 reorganize os blocos de linhas após o MPI_Gatherv, fazendo com que o tempo de comunicação (entre 0,073s e 0,115s na Tabela 1) fosse cerca do dobro do observado na block (0,050s).

**Dynamic (mestre-trabalhador):** Foi a estratégia mais eficiente com 12 processos, alcançando os menores tempos tanto no caso padrão (1,142s e Speedup de 6,12 com dynamic,32, na Tabela 1) quanto no caso de desbalanceamento (0,522 s e Speedup de 5,82 com dynamic,8, na Tabela 3). Como os processos trabalhadores solicitam novas faixas à medida que terminam as anteriores, a diferença de custo entre as regiões da imagem é compensada em tempo de execução. Nas Figuras 5 e 16, as curvas de carga mínima, média e máxima da estratégia dynamic ficam praticamente sobrepostas para chunks de 1 a 32. O tamanho do chunk controla o compromisso entre comunicação e balanceamento:

Com chunk 1, o balanceamento é excelente (FBC de 0,020 na Tabela 1 e 0,057 na Tabela 3), mas cada uma das 4096 linhas é enviada e recebida individualmente, gerando o maior tempo de comunicação dentro da estratégia dynamic (0,096s na Tabela 1 e 0,048s na Tabela 3);

Com chunks 8 e 32, a quantidade de trocas de mensagens cai de forma acentuada, reduzindo o tempo de comunicação para 0,048s e 0,029s no caso padrão (Tabela 1) e para 0,015s e 0,012s no desbalanceamento (Tabela 3), mantendo o FBC baixo e entregando os melhores tempos de execução;

Com chunk 128, o tempo de comunicação é o menor (0,008s na Tabela 1 e 0,006s na Tabela 3), mas os blocos ficam grandes demais para manter todos os 11 trabalhadores ocupados até o final, elevando o FBC para 0,323 no caso padrão e 0,392 no desbalanceamento e aumentando o tempo total (1,315s e 0,662s).

Em contrapartida, a dynamic possui a implementação mais complexa, pois exige controle de envio e recebimento de mensagens ponto a ponto sob demanda e dedica o rank 0 apenas à coordenação. Com poucos processos essa perda pesa: na Tabela 2 e na Figura 8, com 4 processos (apenas 3 trabalhadores calculando), a cyclic,1 foi mais rápida que a dynamic,1 (2,068s contra 3,114s). A partir de 6 processos, e especialmente com 12 processos, o ganho no balanceamento supera a perda de um processo para o mestre (1,124s na dynamic,1 contra 1,261s na cyclic,1 na Tabela 2). Assim, a estratégia dynamic com chunk entre 8 e 32 linhas mostrou-se a mais adequada para o problema.

### 4.4 Análise do Impacto do Número de Processos no Desempenho

Na análise de escalabilidade forte (Tabela 2 e Figuras 8 a 10), o tempo de execução caiu continuamente com o aumento do número de processos nas estratégias cyclic,1 e dynamic,1, chegando ao menor tempo em 12 processos com a dynamic,1 (1,124s, Speedup de 6,22 e eficiência de 51,80%). Ainda assim, conforme o número de processos cresce, a curva de Speedup se afasta gradualmente do valor ideal (S(n,p) = p) e a eficiência diminui.

Pela Lei de Amdahl, se f é a fração do programa que pode ser paralelizada e (1 - f) é a fração sequencial, o Speedup com p processos é dado por:

$$S(p) = \frac{1}{(1-f)+\frac{f}{p}}$$

Nesta implementação, a etapa que não se divide entre os processos no caso estático é a coleta da imagem no rank 0, que levou entre 0,038s e 0,053s na block (Tabela 2), o que representa cerca de 0,7% do tempo sequencial de 6,989 s ($1-f \approx 0{,}0071$ e $f \approx 0{,}993$). Com esse valor, a Lei de Amdahl permitiria um Speedup de até 11,1 com 12 processos, valor bem superior ao Speedup máximo medido (6,22). Essa diferença mostra que a limitação prática do Speedup não veio apenas da coleta sequencial, mas da soma de três fatores: o desbalanceamento de carga (nas estratégias estáticas), a reserva de um processo exclusivo para o mestre (na estratégia dynamic, onde apenas $p-1$ processos calculam pixels) e as características físicas do hardware utilizado.

Conforme descrito na Seção 2, a máquina possui 12 threads lógicas, mas apenas 8 núcleos físicos, sendo 4 de alta performance (P-Cores com Hyper-Threading) e 4 de eficiência (E-Cores, mais lentos). Portanto, 12 processos MPI não equivalem a 12 processadores físicos iguais e independentes: parte dos processos roda em núcleos de eficiência e parte compartilha os mesmos núcleos físicos. Além disso, processos que aguardam mensagens ou barreiras no Open MPI continuam consumindo CPU. Isso explica dois comportamentos observados na Tabela 2 e na Figura 8: na block, o tempo com 4 processos (4,638s) foi maior que com 2 processos (3,611s); e na dynamic,1, a execução com 2 processos (8,410s, Speedup de 0,83) foi mais lenta que a versão sequencial (6,989s), pois havia apenas 1 trabalhador calculando toda a imagem enquanto dividia a máquina com o processo mestre.

Na escalabilidade fraca (Tabela 4 e Figuras 19 a 22), a resolução da imagem cresceu junto com o número de processos para manter a carga por processo aproximadamente constante. Esse comportamento foi analisado pela Lei de Gustafson e pela relação entre a sobrecarga paralela $T_0$ e a eficiência E:

$$T_0 = p \cdot T(n,p) - T(n)$$

$$E = \frac{1}{\frac{T_0}{T(n)} + 1}$$

Quando a razão $T_0/T(n)$ permanece próxima de constante à medida que o problema e o número de processos crescem juntos, a eficiência também se mantém estável. Foi o que ocorreu entre 4 e 8 processos na Tabela 4: o tempo paralelo ficou praticamente constante entre 12,205s e 12,351s, e a eficiência manteve-se entre 56,40% e 59,50% (Figura 21). Com 12 processos (resolução 14189×14189), o tempo subiu para 13,806s e a eficiência caiu para 49,70%, refletindo tanto o compartilhamento de todas as 12 threads lógicas da máquina quanto o aumento do custo de comunicação (que subiu de 0,038s para 0,393s devido ao volume de cerca de 805 MB da imagem). Mesmo assim, o Speedup na escalabilidade fraca cresceu de forma quase linear de 2,35 (4 processos) para 5,96 (12 processos), conforme mostra a Figura 20.

### 4.5 Discussão de Gargalos Encontrados e Limitações da Implementação

**Custo de comunicação:** No caso padrão (4096×4096), o tempo de comunicação variou entre 0,008s e 0,115s (Tabelas 1 e 2), representando menos de 2% do tempo sequencial (6,989s). Na estratégia block, a coleta com MPI_Gatherv manteve-se estável em torno de 0,04s a 0,05s de 1 a 12 processos ( Figura 12). Na cyclic, o custo subiu para a faixa de 0,073s a 0,115s (Figura 7 e Tabela 1) devido à etapa adicional de reorganização das linhas na memória do rank 0. Já na dynamic, o comportamento confirma a relação vista em aula entre latência e largura de banda: com chunk 1, o envio linha a linha gera muitas mensagens pequenas e o tempo de comunicação foi de 0,096 s no caso padrão e 0,048s no desbalanceamento; ao aumentar o chunk para 32 e 128, o número de mensagens diminui drasticamente e o tempo de comunicação cai para 0,029s e 0,008s no caso padrão (Figura 7) e para 0,012s e 0,006s no desbalanceamento (Figura 18). Na escalabilidade fraca (Tabela 4), com o aumento da resolução até 14189×14189 (cerca de 805 MB), o volume de dados transmitidos passa a dominar e o tempo de comunicação sobe para 0,393s.

**Sobrecarga do mestre:** Na estratégia dynamic, o processo de rank 0 atua exclusivamente na distribuição de tarefas e recebimento dos resultados, sem calcular pixels. Nas Tabelas 1, 2 e 3, a coluna Mestre (s) mostra que o mestre permanece ativo durante todo o tempo total de execução (por exemplo, 1,164s de mestre para 1,164s de tempo total em dynamic,1 na Tabela 1), pois ele fica aguardando e atendendo mensagens até o último trabalhador terminar. Na prática, isso retira 1/p dos processos do cálculo do fractal: com 2 processos, metade dos processos (50%) não calcula pixels, causando a perda de desempenho vista na Tabela 2 (Speedup de 0,83); com 12 processos, o mestre representa apenas 1 de 12 processos (8,3%), e o ganho obtido com o balanceamento dinâmico compensa essa perda.

**Tempo ocioso e desbalanceamento:** Nas estratégias estáticas (block e cyclic), os processos que recebem faixas de escape rápido terminam cedo e ficam ociosos aguardando os processos mais lentos na barreira antes da coleta. Esse efeito é severo

na block (FBC de 0,997 no caso padrão e 0,719 no desbalanceamento) e ainda presente na cyclic com chunks maiores (FBC de 0,452 no caso padrão e 0,511 no desbalanceamento com chunk 128).

**Granularidade das tarefas:** A granularidade expressa a razão entre o tempo de computação e o tempo de comunicação. Nas estratégias cyclic e dynamic, chunks muito pequenos (chunk 1) oferecem granularidade fina e ótimo balanceamento, mas aumentam a quantidade de mensagens; chunks muito grandes (chunk 128) oferecem granularidade grossa e pouca comunicação, mas prejudicam o balanceamento (Figuras 6 e 17). Os chunks intermediários (8 e 32 linhas) proporcionaram o melhor equilíbrio.

**Escrita em disco:** Conforme apresentado na Seção 3.2, a gravação dos arquivos de saída (.bin e .ppm) levou 56,08s, quase oito vezes o tempo do cálculo sequencial (6,989s), devido à escrita da imagem PPM em formato de texto. Embora esse tempo tenha sido isolado das medições de desempenho para não distorcer o Speedup, ele representa a etapa mais demorada na execução completa do programa.

**Variação do ambiente de execução:** Por rodar sob o WSL 2 no Windows 11 e sem fixação de processos aos núcleos (binding), a alocação dos processos entre núcleos de performance e de eficiência sofre variação entre diferentes baterias de teste. Por exemplo, a mesma configuração (block com 12 processos no caso padrão) registrou 3,031s na bateria da Tabela 1 e 3,692s na bateria da Tabela 2. A execução de 5 repetições com escolha da mediana reduziu o impacto dessas oscilações em cada bateria.

### 4.6 Comparação com a Solução OpenMP

Conforme mostrado na Seção 3.4.4 e na Tabela 5, a solução OpenMP da Etapa 1 obteve tempos de execução menores que a solução MPI na mesma máquina em todas as quantidades de unidades de processamento testadas. Com 12 unidades e configuração dynamic,1, o OpenMP levou 1,035s (Speedup de 6,49 e eficiência de 54,06%), enquanto o MPI levou 1,124s na dynamic,1 (Speedup de 6,22 e eficiência de 51,80%) e 1,261 s na cyclic,1. Comparando a melhor configuração de cada paradigma com 12 unidades, o OpenMP atingiu 0,946s (dynamic,32) contra 1,142 s do MPI (dynamic,32) no caso padrão, e 0,492s (dynamic,1) contra 0,522s do MPI (dynamic,8) no caso de desbalanceamento.

Essa diferença decorre da arquitetura de cada modelo. No OpenMP, todas as threads compartilham o mesmo espaço de endereçamento, calculam pixels e escrevem diretamente na matriz final em memória compartilhada. No MPI, cada processo possui memória própria, exigindo cópia e envio dos resultados por mensagens até o rank 0, além de reservar um processo inteiro para o papel de mestre na estratégia dynamic. Por isso, a desvantagem do MPI é muito grande com poucas unidades, com 2 unidades na Tabela 5, o OpenMP levou 3,575s (Speedup de 1,88) enquanto o MPI dynamic,1 levou 8,410s (Speedup de 0,83, por ter apenas 1 trabalhador), e diminui bastante quando se chega a 12 unidades, onde o tempo do MPI dynamic,1 ficou apenas 8,6% acima do OpenMP (1,124s contra 1,035s).

Em relação ao comportamento das estratégias, chega-se a uma mesma conclusão: a divisão estática em blocos contíguos apresenta forte desbalanceamento de carga devido à concentração de iterações no centro do fractal, enquanto a distribuição dinâmica sob demanda resolve o problema e entrega os menores tempos de execução. A principal vantagem do MPI não aparece quando executado em um único computador, mas sim na sua capacidade de escalar a execução para múltiplos computadores (nós de um cluster) em memória distribuída, onde o OpenMP sozinho não pode operar. Em contrapartida, o MPI exige maior esforço de programação para dividir o domínio, gerenciar mensagens e montar a imagem final, enquanto no OpenMP essa distribuição é abstraída pela cláusula schedule.

## 5. Conclusão

A realização da Etapa 2 demonstrou, na prática, como o paralelismo em memória distribuída com MPI exige que o programador trate explicitamente tanto a decomposição dos dados quanto o balanceamento de carga e a coleta dos resultados. Embora o cálculo do Conjunto de Mandelbrot não possua dependência entre os pixels, a estratégia block (divisão estática em faixas contíguas) praticamente não escalou além de 2 processos, atingindo FBC de 0,997 e eficiência de apenas 19,21% com 12 processos no caso padrão (Tabela 1). A estratégia cyclic melhorou significativamente a distribuição ao intercalar as linhas (chegando a 1,239s e Speedup de 5,64 com chunk 8), mas continuou limitada por definir a divisão antes da execução e pelo custo extra de reorganizar as linhas no rank 0. O esquema mestre-trabalhador (dynamic) foi o único capaz de manter o FBC próximo de zero de forma consistente, alcançando os melhores resultados gerais com 12 processos: 1,142s (Speedup de 6,12) com dynamic,32 no caso padrão e 0,522s (Speedup de 5,82) com dynamic,8 no caso de desbalanceamento.

A análise do custo de comunicação mostrou que a escolha da granularidade (chunk) é fundamental no esquema dinâmico: chunks entre 8 e 32 linhas equilibraram o baixo número de mensagens com uma distribuição de carga uniforme, evitando tanto o excesso de mensagens pequenas do chunk 1 quanto o desbalanceamento final do chunk 128. Além disso, verificou-se que a coleta da imagem representou menos de 2% do tempo sequencial no caso padrão, sendo os principais limitadores de desempenho a reserva de um processo exclusivo para o mestre na estratégia dynamic, o tempo ocioso nas estratégias estáticas e a arquitetura híbrida de 8 núcleos físicos (12 threads lógicas) da máquina de testes.

Por fim, na comparação direta em uma única máquina (Tabela 5), a solução OpenMP da Etapa 1 manteve tempos menores que a solução MPI (0,946s contra 1,142s nas melhores configurações do caso padrão), pois opera em memória compartilhada sem necessidade de troca de mensagens nem de um processo dedicado à coordenação. Ainda assim, a implementação em MPI cumpriu integralmente os objetivos propostos, reproduzindo com exatidão o resultado da versão sequencial, alcançando acelerações de até 6,22 vezes e permitindo avaliar experimentalmente os compromissos entre computação, comunicação e balanceamento de carga em memória distribuída.

## 6. Referências

ORELLANA, Esbel Tomás Valero. Aula 02 - Introdução ao Processamento Paralelo. Material de aula da disciplina DEC107 – Processamento Paralelo. Universidade Estadual de Santa Cruz, 2026.

ORELLANA, Esbel Tomás Valero. Aula 03 - Caracterização de desempenho em programas paralelos. Material de aula da disciplina DEC107 – Processamento Paralelo. Universidade Estadual de Santa Cruz, 2026.

ORELLANA, Esbel Tomás Valero. Aula 10 - MPI, Introdução e Aplicações Práticas. Material de aula da disciplina DEC107 – Processamento Paralelo. Universidade Estadual de Santa Cruz, 2026.

ORELLANA, Esbel Tomás Valero. Aula 11 - MPI, Comunicações Coletivas. Material de aula da disciplina DEC107 – Processamento Paralelo. Universidade Estadual de Santa Cruz, 2026.

ORELLANA, Esbel Tomás Valero. MPI: Scatter e Gather com o Filtro Gaussiano. Material de aula da disciplina DEC107 – Processamento Paralelo. Universidade Estadual de Santa Cruz, 2026.

MESSAGE PASSING INTERFACE FORUM. MPI: A Message-Passing Interface Standard. Disponível em: https://www.mpi-forum.org/docs/. Acesso em: 30 out. 2026.

OPEN MPI. Open MPI Documentation. Disponível em: https://docs.open-mpi.org/. Acesso em: 30 out. 2026.

# DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Lucas Gabriel Ferreira, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Etapa 2 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação paralela em memória distribuída com MPI", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

( ) Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

**Detalhamento do Uso (se aplicável):** As ferramentas utilizadas foram: Google Gemini e ChatGPT. O uso se deu exclusivamente nas seguintes etapas do projeto:

- compreensão e esclarecimento de conceitos relacionados ao algoritmo de Mandelbrot, programação paralela e MPI;
- sugestão de possíveis melhorias e alternativas para a implementação;
- auxílio na identificação e depuração de erros encontrados durante o desenvolvimento e os testes;
- revisão e aprimoramento da redação de trechos do relatório.

**Termo de Responsabilidade:** Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus — BA, 30 de outubro de 2026

Lucas Gabriel Ferreira
Matrícula: 202311126

# DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Laisa Pereira França, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Etapa 2 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação paralela em memória distribuída com MPI", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

( ) Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

**Detalhamento do Uso (se aplicável):** As ferramentas utilizadas foram: Claude e ChatGPT. O uso se deu exclusivamente nas seguintes etapas do projeto:

- compreensão e esclarecimento de conceitos relacionados ao algoritmo de Mandelbrot, programação paralela e MPI;
- sugestão de possíveis melhorias e alternativas para a implementação;
- auxílio na identificação e depuração de erros encontrados durante o desenvolvimento e os testes;
- elaboração do rascunho e revisão da redação do relatório.

**Termo de Responsabilidade:** Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus — BA, 30 de outubro de 2026

Laisa Pereira França
Matrícula: 202311124